

Bitcoin Blood

Le réseau intelligent et vérifiable des donneurs de sang en Afrique

« *Trouver le bon donneur, au bon endroit, au bon moment* »

Document de référence hackathon : pitch complet (6 fonctionnalités), architecture technique Bitcoin (OpenTimestamps + BIP-322), code de référence, et plan de répartition du travail entre agents IA (Claude / Gemini) pour livrer une démo fonctionnelle.

Note de cadrage : les 6 briques sont conservées pour le pitch et la vision produit. La section 5 indique précisément lesquelles seront réellement codées en 24h et lesquelles restent au stade narratif/mock pour la démo — à ajuster en cours de journée selon l'avancement.

1. Le problème

Chaque année, des milliers de patients perdent la vie en Afrique faute de trouver du sang compatible à temps.

- Les banques de sang sont insuffisamment interconnectées.
- Il n'existe pas de registre régional fiable des donneurs volontaires.
- Les campagnes de collecte atteignent difficilement les bonnes personnes.
- Les donneurs réguliers sont peu valorisés.
- Les certificats ou cartes de donneur sont souvent papier et facilement falsifiables.
- Les données sont dispersées entre plusieurs établissements.

2. La solution

Bitcoin Blood est une plateforme panafricaine qui connecte centres nationaux de transfusion, hôpitaux, ONG, associations de donneurs et donneurs volontaires dans un réseau sécurisé, intelligent et vérifiable.

3. Les 6 fonctionnalités

3.1 Registre numérique des donneurs

Chaque donneur possède un profil sécurisé : groupe sanguin, facteur Rhésus, ville, âge, disponibilité, historique des dons, critères médicaux d'éligibilité. Les données sensibles restent protégées (jamais stockées en clair sur la blockchain), tandis que leur intégrité est garantie par des preuves cryptographiques (OTS).

3.2 Recherche intelligente en urgence

Un hôpital choisit le groupe sanguin recherché, la quantité nécessaire et le lieu. Le système identifie instantanément les donneurs compatibles les plus proches et disponibles, via un algorithme de matching (compatibilité ABO/Rhésus + distance + disponibilité).

3.3 Alertes géolocalisées

Les notifications ne sont envoyées qu'aux donneurs concernés selon le groupe sanguin et un rayon géographique, par exemple : « Besoin urgent de deux donneurs O- dans un rayon de 15 km autour du CNHU ». Cela évite de solliciter inutilement l'ensemble des utilisateurs.

3.4 Carte Bitcoin Blood

Chaque donneur dispose d'une carte numérique et d'une carte physique avec QR Code sécurisé — et, si une carte NFC vierge est disponible, d'une variante NFC scannable au contact d'un téléphone (voir section 3.4 bis). Le QR Code ou le tag NFC permet de vérifier que la carte n'a pas été falsifiée et correspond bien à un enregistrement signé par l'adresse Bitcoin du donneur (groupe sanguin, statut, historique selon les règles de confidentialité définies). L'authenticité de la carte est vérifiée grâce à une preuve cryptographique (signature BIP-322 + ancrage OTS).

Précision importante pour le pitch : la carte prouve qu'un enregistrement n'a pas été altéré et qu'il a été signé par une clé donnée — elle ne prouve pas une identité civile en tant que telle, sauf si un registre séparé (protégé) lie cette clé à une identité réelle.

3.4 bis. Variante carte NFC physique

Une carte NFC vierge (type NTAG213/215/216) permet une démo physique plus impressionnante qu'un simple QR affiché à l'écran : on approche la carte du téléphone et la vérification se lance directement.

- La carte NFC ne fait que transporter les mêmes données que le QR Code (adresse Bitcoin, hash du profil, référence à la preuve OTS) — elle ne signe rien elle-même, la sécurité cryptographique reste entièrement logicielle (BIP-322 + OTS), inchangée par rapport à la version QR.
- Écriture sur la carte via l'API Web NFC (NDEFReader), disponible sur Chrome pour Android uniquement.
- Limite à annoncer à l'oral : Web NFC n'est pas supporté sur iPhone/Safari (restriction Apple). La démo doit se faire sur un téléphone Android ; prévoir le QR Code en repli si un membre du jury veut tester sur iPhone.
- Effort d'intégration faible : remplace simplement l'étape "génération QR" par "écriture NFC", le reste du flux de vérification ne change pas.

3.5 Organisation des campagnes de don

Les centres de transfusion peuvent créer une campagne, définir une localisation, cibler certains groupes sanguins, suivre les inscriptions et mesurer les résultats.

3.6 Programme de fidélisation

Chaque don validé génère des points, débloquant badges et récompenses partenaires. Pour la démo hackathon, on peut illustrer l'envoi de petites récompenses symboliques en satoshis via Lightning Network (réel si le temps le permet, simulé sinon — voir section 5).

4. Pourquoi Bitcoin

Le projet ne stocke jamais les données médicales sur la blockchain. Bitcoin sert uniquement à :

- Garantir que les cartes de donneurs n'ont pas été falsifiées (signature BIP-322 liée à l'adresse Bitcoin du donneur).
- Horodater des preuves cryptographiques attestant de l'intégrité des registres (OpenTimestamps, ancrage gratuit sur la chaîne Bitcoin).
- Renforcer la confiance entre institutions sans dépendre d'une base de données centrale unique.
- Permettre, si souhaité, des récompenses instantanées via le Lightning Network.

4.1 Schéma du flux technique

Étape	Action	Outil Bitcoin
1. Inscription	Génération d'une vraie paire de clés / adresse Bitcoin côté client, jamais transmise au serveur	bitcoinjs-lib
2. Enregistrement du profil	Hash SHA-256 du profil donneur (groupe sanguin, ville, etc.)	—
3. Signature	Le donneur signe le hash avec sa clé privée	BIP-322
4. Ancrage	Le hash signé est envoyé à un calendar server pour horodatage immuable	OpenTimestamps
5. Carte / QR Code	Le QR encode adresse + hash + preuve, scannable pour vérification	—
6. Vérification publique	Recalcul du hash, vérification signature + preuve OTS	BIP-322 + OTS

5. Scope réaliste pour 24h

Les 6 briques restent dans le pitch et la vision produit. Voici la priorisation suggérée pour savoir quoi coder réellement, quoi simuler (mock convaincant pour la démo), et quoi garder uniquement à l'oral — à ajuster selon votre avancement réel dans la journée.

Brique	Statut suggéré J1	Détail
1. Registre numérique des donneurs	À coder	Cœur du projet : formulaire + hash + signature + stockage backend
2. Recherche intelligente en urgence	À coder (version simple)	Matching par compatibilité ABO/Rhésus + distance ; pas de vrai ML, algorithme de scoring clair
3. Alertes géolocalisées	Mock pour la démo	Simuler l'envoi (ex: liste affichée + notification factice) plutôt qu'un vrai système SMS/push
4. Carte Bitcoin Blood + QR	À coder	Génération QR + page de vérification publique : c'est la démonstration la plus impressionnante pour le jury
4 bis. Variante carte NFC	À coder si carte dispo (effort faible)	Web NFC (Chrome Android) en complément du QR ; même logique de vérification, juste un autre support de transport
5. Organisation des campagnes	Oral / maquette visuelle	Quelques écrans statiques (Figma-like) suffisent, pas besoin de vraie logique backend
6. Fidélisation + récompenses sats	Mock (afficher des sats, pas de vraie tx Lightning)	Simuler l'affichage "+500 sats reçus" sans implémenter de vrais canaux Lightning

Recommandation : concentrez l'énergie de codage sur les briques 1, 2 et 4 — c'est ce qui démontre concrètement l'usage de Bitcoin (le cœur du sujet du hackathon). Les briques 3, 5, 6 enrichissent le pitch sans demander le même temps d'ingénierie.

6. Code de référence (JavaScript)

6.1 Génération du wallet donneur (côté client)

```
import * as bitcoin from 'bitcoinjs-lib';
import * as ecc from 'tiny-secp256k1';
import { ECPairFactory } from 'ecpair';

const ECPair = ECPairFactory(ecc);
const keyPair = ECPair.makeRandom();
const { address } = bitcoin.payments.p2wpkh({ pubkey: keyPair.publicKey });
// address = identifiant Bitcoin du donneur, ex: bc1q...
// keyPair.privateKey reste en local, jamais envoyé au backend
```

6.2 Hash du profil + signature BIP-322

```
import { BIP322 } from 'bip322-js';
import { sha256 } from 'js-sha256';

const donorProfile = { bloodType: 'O-', city: 'Cotonou', age: 28, available: true };
const profileHash = sha256(JSON.stringify(donorProfile));
const signature = BIP322.signMessage(profileHash, keyPair.toWIF());
```

6.3 Ancrage immuable via OpenTimestamps

```
import * as OpenTimestamps from 'javascript-opentimestamps';

const hashBuffer = Buffer.from(profileHash, 'hex');
const detached = OpenTimestamps.DetachedTimestampFile.fromHash(
  new OpenTimestamps.Ops.OpSHA256(), hashBuffer
);
await OpenTimestamps.stamp(detached);
const otsProof = detached.serializeToBytes();
// otsProof à stocker, encodé dans le QR Code de la carte
```

6.4 Matching simple pour la recherche d'urgence

```
// Compatibilité ABO/Rhésus simplifiée + tri par distance
const compatibility = {
  'O-': ['O-'], 'O+': ['O-', 'O+'],
  'A-': ['O-', 'A-'], 'A+': ['O-', 'O+', 'A-', 'A+'],
  'B-': ['O-', 'B-'], 'B+': ['O-', 'O+', 'B-', 'B+'],
  'AB-': ['O-', 'A-', 'B-', 'AB-'], 'AB+': ['O-', 'O+', 'A-', 'A+', 'B-', 'B+', 'AB-', 'AB+'],
};

function findMatchingDonors(requestedType, hospitalCoords, donors) {
  return donors
    .filter(d => compatibility[requestedType].includes(d.bloodType) && d.available)
    .map(d => ({ ...d, distanceKm: haversine(hospitalCoords, d.coords) }))
    .sort((a, b) => a.distanceKm - b.distanceKm)
    .slice(0, 10);
}
// Volontairement appelé "algorithme de matching", pas "IA", pour rester défendable techniquement
```

6.5 Écriture NFC (alternative/complément au QR Code)

```
// Web NFC API – Chrome Android uniquement
async function writeCardNFC(address, profileHash, otsProofUrl) {
  const ndef = new NDEFReader();
  const payload = JSON.stringify({ address, profileHash, otsProofUrl });
  await ndef.write({
    records: [{ recordType: "text", data: payload }]
  });
}
// Lecture côté vérificateur
```

```

async function readCardNFC(onRead) {
  const ndef = new NDEFReader();
  await ndef.scan();
  ndef.onreading = (event) => {
    const decoder = new TextDecoder();
    for (const record of event.message.records) {
      const data = JSON.parse(decoder.decode(record.data));
      onRead(data); // { address, profileHash, otsProofUrl }
      // -> reste du flux identique à la vérification QR (section 6.6)
    }
  };
}

```

6.6 Vérification publique (scan QR ou NFC de la carte)

```

const isSignatureValid = BIP322.verifySignature(profileHash, address, signature);

const detached = OpenTimestamps.DetachedTimestampFile.deserialize(otsProofBytes);
const verifyResult = await OpenTimestamps.verify(detached, hashBuffer);

if (isSignatureValid && verifyResult.bitcoin) {
  console.log(`Carte authentique, signée par ${address}, ` +
    `horodatée avant le bloc ${verifyResult.bitcoin.height}`);
}

```

Librairies npm : bitcoinjs-lib, tiny-secp256k1, ecpair, bip322-js,
 javascript-opentimestamps, js-sha256, qrcode (génération QR).

7. Plan de sprint — 2h avec Claude + Gemini

Répartition en sous-agents spécialisés pour paralléliser le travail sur les briques 1, 2 et 4 (celles codées réellement, cf. section 5), plus les mocks pour 3 et 6, et la maquette pour 5.

Sous-agent	Outil IA suggéré	Mission (15-30 min)	Livrable attendu
Agent 1 — Wallet & Signature	Claude (rigueur sur specs BIP-322)	Génération clé bitcoins-lib + signature/vérification BIP-322 du profil donneur	Module JS : createWallet(), signProfile(), verifySignature()
Agent 2 — Ancrage OTS	Claude (gestion async/API fiable)	Implémenter stamp() + verify() avec javascript-opentimestamps	Module JS : stampHash(), verifyTimestamp()
Agent 3 — Matching d'urgence	Gemini ou Claude (logique métier simple)	Implémenter le matching ABO/Rhésus + tri par distance (section 6.4)	Fonction findMatchingDonors() + jeu de données donneurs fictifs
Agent 4 — Backend / stockage	Gemini (rapide pour boilerplate CRUD/API REST)	API : POST /donors (profil+sig+adresse+preuve), GET /search, GET /verify/:id	Backend fonctionnel avec routes + base légère (SQLite/Firestore)
Agent 5 — UI/UX + QR/NFC + mocks	Gemini (génération rapide de composants visuels)	Front : inscription donneur, génération QR + écriture NFC (Web NFC, section 6.5) sur la carte, page recherche urgence, page vérification publique (QR ou NFC), mocks alertes/fidélisation	Front HTML/React connecté aux modules des agents 1-4, démo NFC sur Android si carte disponible
Intégrateur (toi)	—	Brancher les modules, tester le flux complet, préparer le pitch (incluant les briques 3, 5, 6 à l'oral)	Démo fonctionnelle + script de présentation 2-3 min

7.1 Timeline suggérée

0:00 - 0:10	Briefing : chaque agent reçoit sa mission + le code de référence (section 6) comme base
0:10 - 1:10	Les 5 agents travaillent en parallèle (conversations Claude/Gemini lancées en simultané)
1:10 - 1:35	Intégration : assemblage des modules, correction des incompatibilités d'interface
1:35 - 1:50	Test du flux complet : inscription → signature → ancrage → matching → vérification QR
1:50 - 2:00	Préparation du pitch : capture d'écran/vidéo de secours, répartition de la parole pour les 6 briques

7.2 Conseil pratique

Donne à chaque agent la section de code correspondante (section 6) comme point de départ littéral, et impose les noms de fonctions du tableau ci-dessus — l'intégration finale doit être un branchement, pas une réécriture. Pour le pitch, sois transparent sur ce qui est codé en dur vs simulé : un jury technique valorise plus une démo honnête et fonctionnelle qu'une promesse non tenue.